University of Antwerp
Algorithms and Complexity
Teaching assistant: Tamara Cucumides

Exercises session 2

Non-deterministic turing machines,

---

1 Definitions

Definition 1.1 (Non-deterministic Turing Machine) *A non-deterministic turing machine is defined as* $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ *where*

- Q *is a finite set of states*
- Σ *is the input alphabet, with* $\vdash, B \notin \Sigma$
- Γ *is the tape alphabet, with* $\Sigma \cup \{\vdash, B\} \subseteq \Gamma$
- $q_0 \in Q$ *is the initial state*
- $q_{accept}, q_{reject} \in Q$ *are the accepting and rejecting states*
- δ *is a* **relation***:*

$$\delta \quad \subseteq \quad Q \times \Gamma \times Q \times \Gamma \times \{\leftarrow, \square, \rightarrow\}$$

Definition 1.2 *Let* $t : \mathbb{N} \to \mathbb{N}$ *be a function. We define the (deterministic)* **time complexity class** $DTIME(t(n))$ *as:*

$$DTIME(t(n)) = \{L \mid L \text{ is decided by an } O(t(n)) \text{ time deterministic Turing machine}\}.$$

Definition 1.3 *PTIME (or* **P***) is the class of all languages decidable in deterministic polynomial time on a single-tape TM*

$$PTIME = \bigcup_k DTIME(n^k).$$

2 Session exercises

1. Lets introduce (recall?) boolean formulas
   - **Syntax of boolean formulas**: let's consider a set of variables **Vars**: $\{x_1, \ldots, x_n\}$ and operators $\vee, \wedge, \neg, \rightarrow, \leftrightarrow$, then we define boolean formulas recursively as
     - every variable x_i is a boolean formula

- if φ and ϕ are boolean formulas, then $(\neg\varphi)$, $(\varphi \wedge \phi)$, $(\varphi \vee \phi)$, $(\varphi \rightarrow \phi)$ and $(\varphi \leftrightarrow \phi)$ are also boolean formulas

- **Semantics of boolean formulas**: a truth assignment σ maps each variable x_i to either **true** (1) or **false** (0), formally $\sigma : \textbf{Vars} \rightarrow \{0, 1\}$. We can evaluate a boolean formula ϕ using a truth assignment σ: we will denote this as $\phi(\sigma)$
  - the evaluation of a variable x_i corresponds to $\sigma(x_i)$
  - if ϕ_1 and ϕ_2 are boolean formulas, then if :
    * $\varphi = (\neg\phi)$, then $\varphi(\sigma) = 1 - \phi(\sigma)$
    * $\varphi = (\phi_1 \wedge \phi_2)$ then $\varphi(\sigma) = \min\{\phi_1(\sigma), \phi_2(\sigma)\}$
    * $\varphi = (\phi_1 \vee \phi_2)$ then $\varphi(\sigma) = \max\{\phi_1(\sigma), \phi_2(\sigma)\}$
    * $\varphi = (\phi_1 \rightarrow \phi_2)$ then $\varphi(\sigma) = 1$ iff $\phi_1(\sigma) = 0$ or $\phi_2(\sigma) = 1$
    * $\varphi = (\phi_1 \leftrightarrow \phi_2)$ then $\varphi(\sigma) = 1$ iff $\phi_1(\sigma) = \phi_2(\sigma)$
  - **Satisfiability**: we say a boolean formula φ is satisfiable, if there exists a truth assignment σ such that $\varphi(\sigma) = 1$
  - **Conjunctive normal form**: we say a boolean formula is in conjunctive normal form (or CNF) if its a conjunction of disjunctions of literals. A literal is a variable or a negation of a variable. An example:

$$\varphi = (x_1 \vee \neg x_2) \wedge (x_1 \vee x_3)$$

In this exercise we will introduce the CNF-SAT problem, that will play an important role in future lessons of this course.

(a) Given a CNF boolean formula φ and an assignment σ for such formula, give a TM that *verifies* whether $\varphi(\sigma) = 1$. What is the running time of such algorithm?

(b) Now lets consider the following decision problem

$$\text{CNF-SAT} = \{\varphi : \varphi \text{ is a satisfiable CNF boolean formula } \}$$

Can we use the idea of the question above to solve this? What would be the running time?

(c) What if we try using non-deterministic turing machines?

2. Show that the following language

$$\text{TRIANGLE} = \{\langle G \rangle : G \text{ has a triangle } \}$$

is in **P**

3. Analize the following algorithm

   Select a node in G and mark it
   while no new nodes are marked **do**
      for each unmarked node u in G **do**
         mark u if it's connected to a marked node
      end for
   end while
   If all nodes are marked, return `TRUE`, otherwise, return `FALSE`

   and conclude that

   $$\text{CONNECTED} = \{\langle G \rangle : G \text{ is a connected undirected graph } \}$$

   is in **P**

4. Let's G be an undirected graph. A path is considered simple if it doesn't contain repeated nodes.

   - Prove that

     $$\text{SPATH} = \{\langle G, a, b, k \rangle : \text{ there's a simple path from } a \text{ to } b \text{ of length at most } k\}$$

     is in **P**

   - Is the following language also in **P**?

     $$\text{LPATH} = \{\langle G, a, b, k \rangle : \text{ there's a simple path from } a \text{ to } b \text{ of length at least } k\}$$

5. Let's introduce the SUBSET-SUM problem: given a set of integers $\mathcal{A} = \{a_1, \ldots, a_n\}$ and another integer K represented in binary. Is there a subset of $\mathcal{A}$ that sums exactly K?

   - Is this language in **P**?
   - Now let's think of the same problem, but the integers are represented in unary. How does our complexity analysis change?

Q1

(a) Given a CNF boolean formula φ and an assignment σ for such formula, give a TM that *verifies* whether $\varphi(\sigma) = 1$. What is the running time of such algorithm?

(b) Now lets consider the following decision problem

$$\text{CNF-SAT} = \{\varphi : \varphi \text{ is a satisfiable CNF boolean formula }\}$$

Can we use the idea of the question above to solve this? What would be the running time?

(c) What if we try using non-deterministic turing machines?

Solutions:

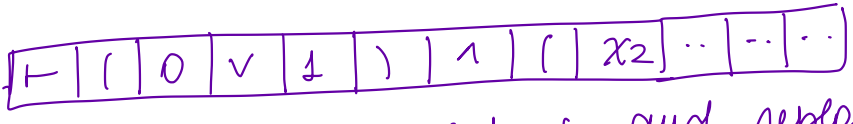
(a) we can use as input, the truth assignment of our formula, and it would look like this

$\varphi = (x_1 \vee x_2) \wedge (x_2 \vee \neg x_3) \wedge (x_1 \vee x_2)$

∇ : $x_1 \rightarrow 0$

$x_2 \rightarrow 1$

$x_3 \rightarrow 0$



we should first remove negations, and replace

$\neg 1 \rightarrow B0$ } we can just ignore B's after

$\neg 0 \rightarrow B1$

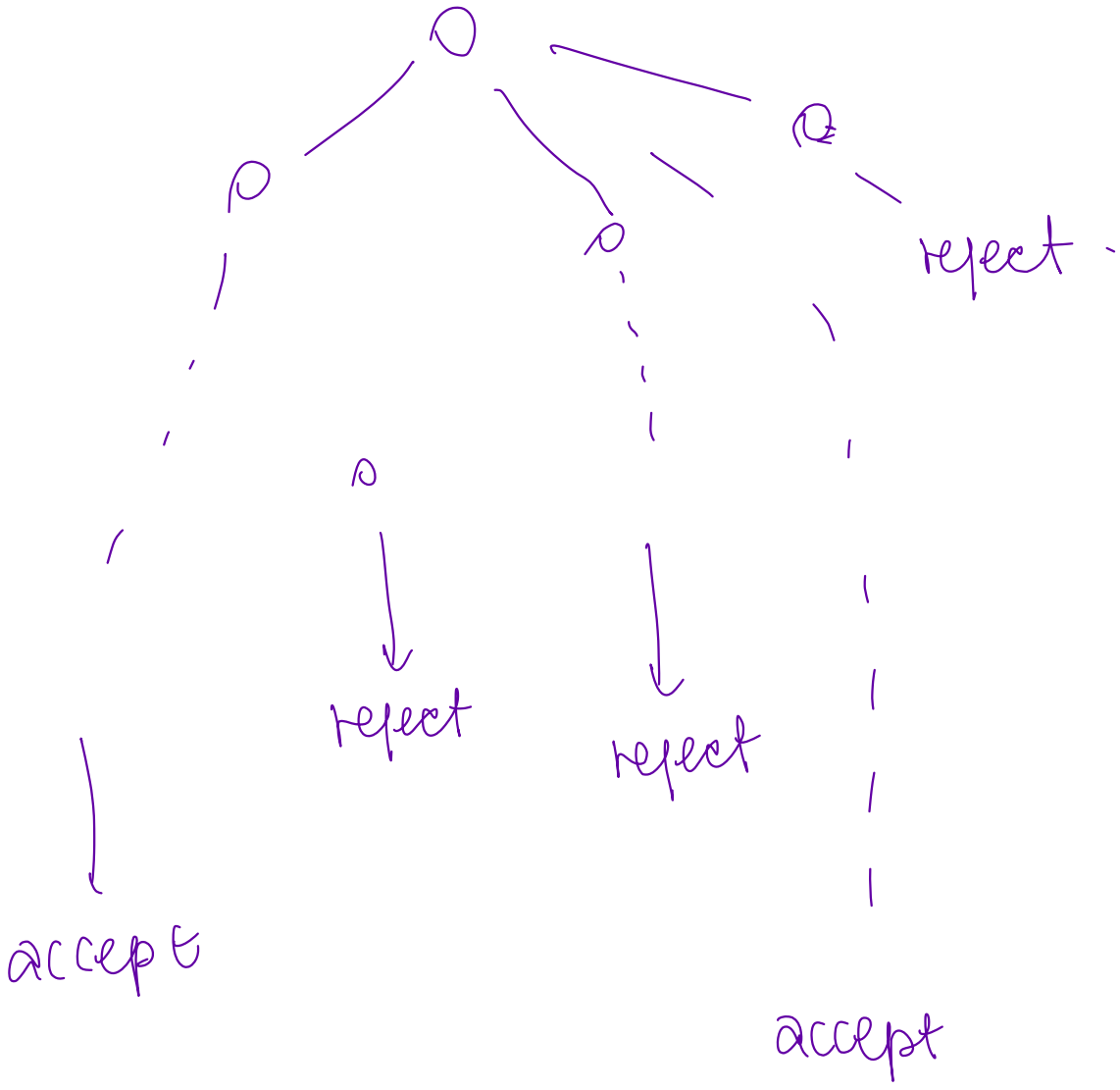
then after we can see if there's at least one 1 between each parenthesis. this should be doable in linear time.

(b) and (c)

Let's make some remarks:

- for a formula φ with variables $x_1, \ldots, x_m$ there are 2^m possible assignments
- run above algorithm for every single assignment doesn't seem like a very << good >> idea (why?)
- what would be the running time of that?
- deciding whether a particular assignment satisfies a formula seems << easy >>
- then deciding if such assignment exists should be easy as well ... or not?

Let's turn our attention into non-determinism ...



- at each step, we can choose / guess which path to take ... how can this help us?

2. Show that the following language

$$\text{TRIANGLE} = \{\langle G \rangle : G \text{ has a triangle } \}$$

is in **P**

given a graph $G = (V, E)$, (undirected)

a "triangle" is a set of vertices

v_1, v_2, v_3 s.t (v_1, v_2), (v_2, v_3), $(v_1, v_3) \in E$

We can decide TRIANGLE as follows

Lets $V = v_1, \ldots, v_n$

1. enumerate all tuples (v_i, v_j, v_k) $(i < j < k)$
   FOR each tuple:
   2. check if $(v_i, v_j), (v_j, v_k), (v_i, v_k) \in E$
   3. if so, accept
4. reject.

1. enumeration is $O(|V|^3)$
2. checking edges takes $O(|E|)$ [or $O(|V|^2)$].

∴ the procedure takes $O(|V|^3 |E|)$ which is polynomial in the size of the input

or $O(|V|^5)$ if we want to keep it all in terms of number of vertices

3. Analize the following algorithm

Select a node in G and mark it
1 **while** no new nodes are marked **do**
2 **for** each node in G mark it if it's connected to a marked node **do**
end for
end while
3 If all nodes are marked, return `TRUE`, otherwise, return `FALSE`

and conclude that

$$\text{CONNECTED} = \{\langle G \rangle : G \text{ is a connected undirected graph } \}$$

is in **P**

Let $G = (V, E)$ and $|V| = n$.

So, step (2) takes $O(m^2)$[1] (the amount of edges), now we need to check how many times we execute that.

In the worst-case scenario, we will mark exactly one node everytime

↓ like this graph

then the while is repeated at most $O(n)$ times

therefore, the running time is $O(n^3)$

TO-DO: check that the algorithm is correct for deciding if the graph is connected.

4. Let's G be an undirected graph.

   - Prove that

     $$\text{SPATH} = \{\langle G, a, b, k\rangle : \text{ there's a simple path from } a \text{ to } b \text{ of length at most } k\}$$

     is in **P**

   - Is the following language also in **P**?

     $$\text{LPATH} = \{\langle G, a, b, k\rangle : \text{ there's a simple path from } a \text{ to } b \text{ of length at least } k\}$$

Let's build an algorithm for SPATH.
We're given $\langle G, a, b, k\rangle$ and say $G=(V,E)$, $|V|=n$
the algorithm goes as follows

(1) mark node a with a 0. (zero)
(2) For $i \in [0, k]$:
  (3) for each edge (u,v), if u is marked with i and v is unmarked, then mark v with $i+1$.
(4) if b is marked with value $\leq k$, accept, otherwise reject.

this is basically doing a BFS
- For goes to k ($k \leq n$)
- each edge is checked one time.

} $O(|V|+|E|)$ or $O(n^2)$

We will see in 2 more sessions that it's very likely that LPATH is not in **P**

5. Let's introduce the SUBSET-SUM problem: given a set of integers $\mathcal{A} = \{a_1, \ldots, a_n\}$ and another integer K represented in binary. Is there a subset of $\mathcal{A}$ that sums exactly K?

   - Is this language in **P**?
   - Now let's think of the same problem, but the integers are represented in unary. How does our complexity analysis change?

Let's build an algorithm for subset sum:
say that $|A| = n$.
How many subsets are in A? 2^n → already exponential.

But... exponential in terms of what?
Ok in the amount of numbers I have,
but... how do I encode such numbers?
Say that the input size in binary is n, then the input size in unary is 2^n.

UNARY → to represent number x, we need x squares in our TM

BINARY → to represent number x, we need $\log x$ squares

now, let's turn into SUBSET SUM again.

We can design a dynamic-programming algorithm for this problem.

(you can check it online, is not very easy to explain on paper)

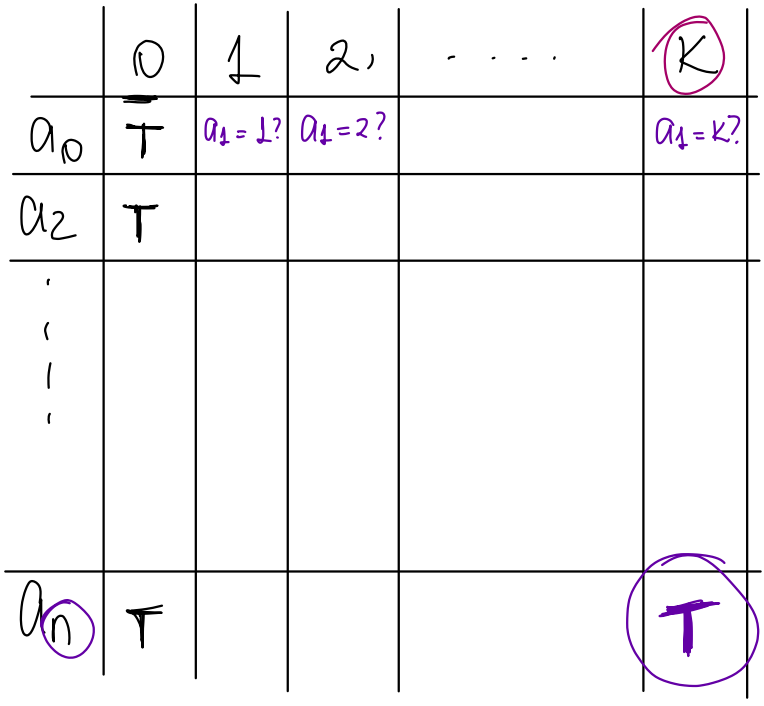
BASIC IDEA:

- sort $a_1, \dots, a_n$ in ascending order
  - for each a_i, we can either include it in the subset or not,
    - if included $\rightarrow$ SUBSET$(A - \{a_i\}, K - a_i)$
    - if not, $\rightarrow$ SUBSET$(A - \{a_i\}, K)$

if you do this in a "smart way"
it takes $O(nK)$

↳ if K is in unary, then this is polynomial in terms of input

↳ if K is in binary, input is $O(\log K)$ so algorithm is exponential...



MATRIX M.

if this is true, we accept

- the 0's column is always true, as you can from 0 with the empty set
- FOR first row, $M[0][j] = 1$ if $a_0 = j$ / $j > 0$
- FOR $i \in [1, \ldots, n]$
  FOR $j \in [1, \ldots, K]$
    IF $a_i < j$:
      $M[i][j] = M[i-1][j]$ # copy from above
    ELSE:
      $M[i][j] = \begin{cases} T & M[i-1][j] = T \\ T & M[i-1][j-a_i] = T \\ F & \sim \end{cases}$